TRABALHO I PI II

Software de aprendizagem mini-MIPS

Ferramenta educativa para explorar **arquitetura de computadores**, **instruções MIPS** e **execução de código** de forma visual e interativa.

NOME:Gabriel More de Lima,Lorezeno Kissmann Rota e Guilherme Macke Ebert

Made with GAMMA

Introdução ao Projeto e Estrutura da Apresentação

Esta apresentação detalhará o desenvolvimento do nosso software de aprendizagem MIPS, abordando desde a concepção até a implementação e funcionamento do produto.

1

Organização do Projeto

Entenda como dividimos as tarefas em 4 semanas, as ferramentas de colaboração e como superamos os desafios de desenvolvimento.

2

O Nosso Produto

Apresentaremos as funcionalidades do software MIPS, guia de utilização e como ele auxilia na interpretação das instruções.

3

Detalhes da Implementação

Descubra como o projeto foi programado, as tecnologias envolvidas e as contribuições individuais na construção de cada módulo.

Made with GAMMA

ORGANIZAÇÃO DO PROJETO

Como o projeto foi organizado e distribuído entre os membros da equipe

Made with GAMMA

Evolução do Projeto

Semana 1: Planejamento Inicial

Definição de escopo, divisão de tarefas e criação da interface do menu.

Semana 2: Decodificação Essencial

Implementação do menu, leitura de instruções e início da decodificação MIPS.

Semana 3: Componentes do Processador

Desenvolvimento de suporte para os tipos R, I, J e construção dos módulos principais.

Semana 4: Funcionalidades Estendidas

Adição de recursos como "voltar", estatísticas de execução e geração de arquivos de saída.

Made with GAMMA

Lorenzo

semana 2

Responsável pela leitura do arquivo de entrada (.txt) contendo instruções em binário e envio para a memória de instruções.

semana 3

Organizou o código, implementou a função de execução e desenvolveu as instruções do tipo R e modularizou mais o programa.

semana 4

Desenvolveu a funcionalidade de *step back*, permitindo ao usuário retornar uma instrução e acompanhar melhor a execução.

Desafios enfrentados por esse integrante: Houve dificuldade inicial para compreender o que precisava ser feito e como executar as tarefas. Além disso, a visão fragmentada do projeto, focada apenas em pequenas partes, dificultou o entendimento do todo.

Made with GAMMA

Guilherme

semana 2

Responsável pela criação do menu de interação com o usuário, garantindo uma experiência intuitiva.

semana 3

Implementou as instruções de transferência de dados (lw e sw)

semana 4

Criou o módulo de estatísticas, responsável por contabilizar e exibir a quantidade de instruções de cada tipo executadas no programaDesafios da Semana

Desafios enfrentados por esse integrante: Disponibilidade de tempo

Made with GAMMA

Gabriel

semana 2

Responsável por converter instruções em binário, separando os campos da instrução e armazenando em uma struct.

semana 3

implementou as instruções de desvio (beq e j), ajustou a ULA e desenvolveu os MUX necessários

semana 4

Implementou a geração dos arquivos de saída, incluindo o arquivo .asm (tradução das instruções para assembly) e o .dat (conteúdo da memória de dados).

Desafios enfrentados por esse integrante: Dificuldade em compreender o projeto e dividi-lo em partes para execução durante as sprints, além de desafios no compartilhamento de arquivos devido à falta de conhecimento no uso do Git.

Made with GAMMA

Software Educacional MINI-MIPS
Guia do Usuário

Made with GAMMA

Simulador MINI-MIPS – Menu Principal

Interface do menu principal do simulador

```
[1] Carregar memoria de instrucao
[2] Carregar memoria de dados
[3] Imprimir memoria de instrucoes e dados
[4] Imprimir banco de registradores
[5] Imprimir todo simulador
[6] Salvar .asm e .dat
[7] Mostrar Estatisticas do programa
[8] Executar programa(RUN)
[9] Executar uma instruçao(STEP)
[10] Voltar uma instrucao
escolha uma opcao:
```

funcionalidades do programa

Nesta etapa, serão apresentadas as opções disponíveis no menu do simulador, destacando a importância de cada uma no processo de aprendizagem.

Cada funcionalidade foi desenvolvida com foco educacional, auxiliando o usuário a compreender, de forma prática e visual, o funcionamento da arquitetura MIPS. Através dessas opções, é possível explorar diferentes aspectos do processamento de instruções, facilitando o entendimento dos conceitos teóricos na prática.

A seguir, serão apresentadas as funcionalidades do menu em detalhes.

Made with GAMMA

Simulador MINI-MIPS – Menu Principal

INTERFACE NO TERMINAL QUANDO REALIZA STEP

```
instrucao em binario:0100000100000001|Assembly: addi $4,$0,1
Instrução executada!
PC da proxima instrucao:2
```

funcionalidades do programa

Opções de Execução

- **Run:** Executa todo o programa de forma automática, sem exibir detalhes ao usuário. Essa opção é útil para obter o resultado final da execução de maneira rápida.
- **Executar uma instrução:** Permite analisar o programa passo a passo. Ao selecionar essa opção, o usuário visualiza a instrução em binário, sua representação em assembly e seus campos já separados (como rs, rt, rd, etc.), facilitando o entendimento de como cada instrução é processada.

Made with GAMMA

Simulador MINI-MIPS – Menu Principal

IMPRIMIR TODO SIMULADOR

```
Imprimindo banco de registradores e memória de dados:
======memoria de dados======
[  0] =   0 |[  1] =   0 |[  2] =   0 |[  3] =   0 |[  4] =   0 |[  5] =   0 |[  6] =   0
[ 16] =   0 |[ 17] =   0 |[ 18] =   0 |[ 19] =   0 |[ 20] =   0 |[ 21] =   0 |[ 22] =   0
[ 32] =   0 |[ 33] =   0 |[ 34] =   0 |[ 35] =   0 |[ 36] =   0 |[ 37] =   0 |[ 38] =   0
[ 48] =   0 |[ 49] =   0 |[ 50] =   0 |[ 51] =   0 |[ 52] =   0 |[ 53] =   0 |[ 54] =   0
[ 64] =   0 |[ 65] =   0 |[ 66] =   0 |[ 67] =   0 |[ 68] =   0 |[ 69] =   0 |[ 70] =   0
[ 80] =   0 |[ 81] =   0 |[ 82] =   0 |[ 83] =   0 |[ 84] =   0 |[ 85] =   0 |[ 86] =   0
[ 96] =   0 |[ 97] =   0 |[ 98] =   0 |[ 99] =   0 |[100] =   0 |[101] =   0 |[102] =   0
[112] =   0 |[113] =   0 |[114] =   0 |[115] =   0 |[116] =   0 |[117] =   0 |[118] =   0
[128] =   0 |[129] =   0 |[130] =   0 |[131] =   0 |[132] =   0 |[133] =   0 |[134] =   0
[144] =   0 |[145] =   0 |[146] =   0 |[147] =   0 |[148] =   0 |[149] =   0 |[150] =   0
[160] =   0 |[161] =   0 |[162] =   0 |[163] =   0 |[164] =   0 |[165] =   0 |[166] =   0
[176] =   0 |[177] =   0 |[178] =   0 |[179] =   0 |[180] =   0 |[181] =   0 |[182] =   0
[192] =   0 |[193] =   0 |[194] =   0 |[195] =   0 |[196] =   0 |[197] =   0 |[198] =   0
[208] =   0 |[209] =   0 |[210] =   0 |[211] =   0 |[212] =   0 |[213] =   0 |[214] =   0
[224] =   0 |[225] =   0 |[226] =   0 |[227] =   0 |[228] =   0 |[229] =   0 |[230] =   0
[240] =   0 |[241] =   0 |[242] =   0 |[243] =   0 |[244] =   0 |[245] =   0 |[246] =   0

=====BANCO DE REGISTRADORES=====
[R0] = 0 [R1] = 0 [R2] = 0 [R3] = 4 [R4] = 1 [R5] = 0 [R6] = 0 [R7] = 0
```

funcionalidades do programa

- **imprimir Memória (Instruções e Dados):** Permite visualizar o conteúdo das memórias. As instruções são exibidas em binário e em assembly, facilitando a associação entre as representações e auxiliando no aprendizado.
- **Imprimir Banco de Registradores:** Exibe os valores atuais dos registradores, permitindo ao usuário acompanhar como as instruções afetam os dados ao longo da execução.
- **Imprimir Todo o Simulador:** Mostra o estado completo do programa, incluindo memória e registradores. Essa opção é útil para entender o impacto geral de uma instrução, como por exemplo após executar um `sw`.
- **Step Back:** Permite retornar uma instrução na execução do programa. Essa funcionalidade é importante para o aprendizado, pois possibilita ao usuário revisar estados anteriores, comparar mudanças e compreender melhor o efeito de cada instrução no sistema.

Made with GAMMA

Simulador MIPS – Menu Principal

ARQUIVO.ASM

```
addi $2,$0,-5
addi $1,$0,3
add $3,$2,$1
add $3,$2,$1
addi $4,$0,10
sw $2,0($0)
lw $1,4($0)
beq $0,$2,8
beq $0,$5,1
beq $3,$2,10
jump 1
```

funcionalidades do programa

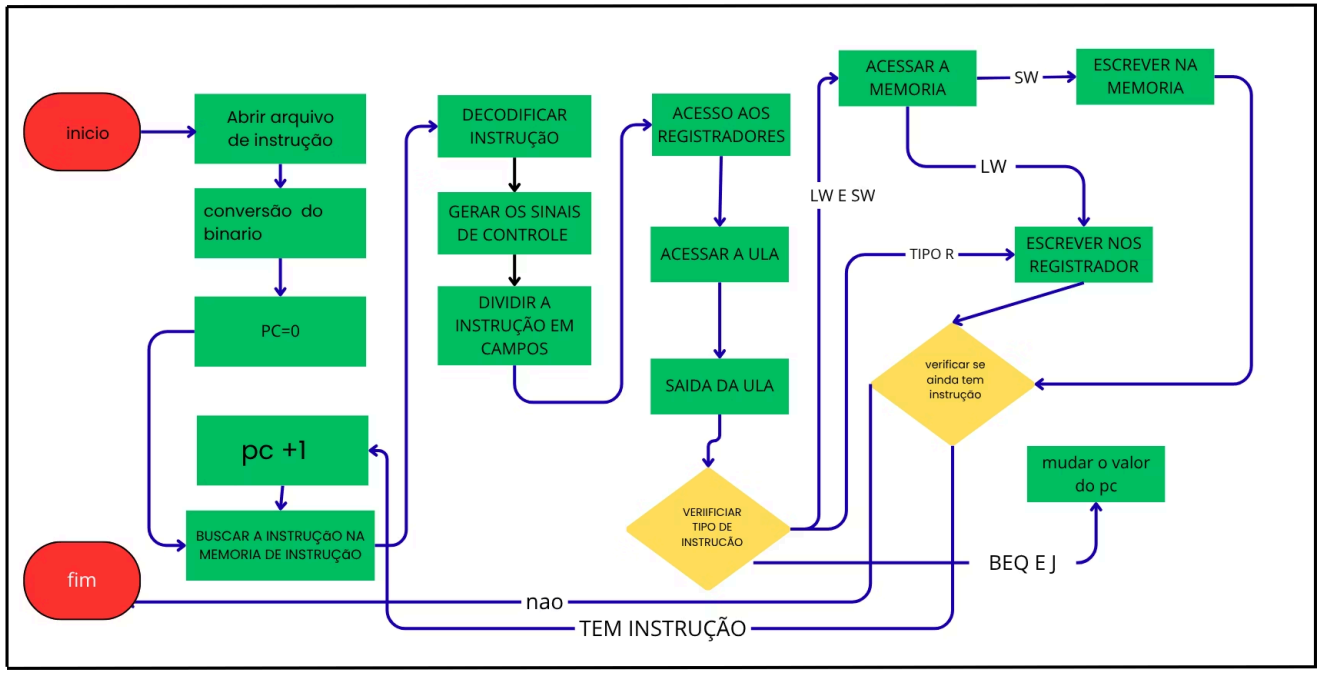
- **Arquivo .dat:** Gera a memória de dados do programa, permitindo ao usuário visualizar os valores armazenados após a execução. Isso ajuda a entender como as instruções manipulam os dados na prática.
- **Arquivo .asm:** Gera o código em assembly a partir das instruções em binário, facilitando a leitura e a interpretação do programa. Essa funcionalidade auxilia na associação entre diferentes representações das instruções, reforçando o aprendizado

Made with GAMMA

IMPLEMENTAÇÃO DO PROJETO
COMO O PROJETO FOI FEITO

Made with GAMMA

FLUXOGRAMA DA IMPLEMENTAÇÃO



Made with GAMMA

PROGRAMA CRIADO PELO GRUPO

PROGRAMA EM C

```c
#include <stdio.h>

int fatorial(int n);

int main() {
    int num=4;
        printf("Fatorial de %d = %d\n", num, fatorial(num));
    return 0;
}

// Função recursiva para calcular fatorial
int fatorial(int n) {
    if (n == 0 || n == 1) {
        return 1;
    }
    return n * fatorial(n - 1);
}
```

Made with GAMMA

PROGRAMA CRIADO PELO GRUPO

INSTRUÇÕES EM BINÁRIO

```
=======MEMORIA DE INSTRUCAO======
Instrução 1: 0100000011000100
Instrução 2: 0100000100000001
Instrução 3: 1000000011001001
Instrução 4: 0000011000101000
Instrução 5: 0100000110000000
Instrução 6: 1000000101000011
Instrução 7: 0000110100110000
Instrução 8: 0100101101111111
Instrução 9: 0010000000000101
Instrução 10: 0000110000100000
Instrução 11: 0100011011111111
Instrução 12: 0010000000000010
Instrução 13: 1111010100000000
```

INSTRUÇÕES EM LINGUAGEM ASSEMBLY

```
1   addi $3,$0,4
2   addi $4,$0,1
3   beq $0,$3,9
4   add $5,$3,$0
5   addi $6,$0,0
6   beq $0,$5,3
7   add $6,$6,$4
8   addi $5,$5,-1
9   jump 5
0   add $4,$6,$0
1   addi $3,$3,-1
2   jump 2
3   sw $4,0($2)
```

Made with GAMMA